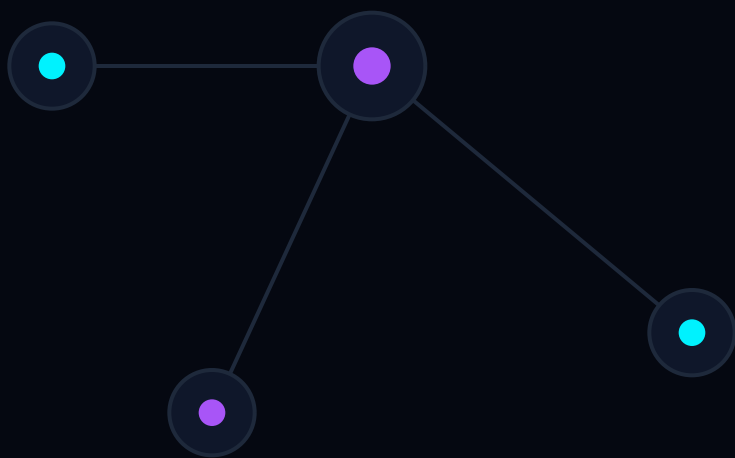




1. Executive Summary & Overview

The **Spatial DSA Tutor (Immersive Learning Lab)** is an interactive WebGL-based educational tool designed to visualize Data Structures and Algorithms (DSA) within a zero-gravity space simulation environment. By combining modern 3D graphics (Three.js), computer vision hand tracking (MediaPipe Hands API), virtual reality capabilities (WebXR), and standard web technologies, the project offers an engaging and highly tactile method for exploring abstract programming concepts.

Unlike traditional static text representations or 2D canvas visualizers, the Spatial DSA Tutor treats data elements as physical objects. Nodes can drift, levitate, bounce, and glide based on simulated gravity and spring physics. Users can interact with these components using three coexisting control systems:

- **Webcam Hand Gestures:** Real-time hand tracking maps the user's hand coordinates to 3D space, allowing users to pinch and drag nodes, zoom using left-hand dollying, and perform dual-hand transformations (scaling and rotation).
- **Desktop Cyber-Glove Simulator:** A physics-based mouse and keyboard fallback system that simulates a 3D hand in the classroom, rendering a virtual glove to maintain the zero-gravity interaction experience even without a webcam.
- **Physical Controls & Shortcuts:** Standard keyboard hotkeys (Space for Play/Pause, ArrowRight/Left for steps) and physical mouse clicks/draggs to directly control orbit camera systems and drag blocks.

2. Technical Stack & Dependencies

The project leverages lightweight, high-performance, and client-side dependencies to achieve real-time, low-latency, and interactive WebGL experiences without needing a heavyweight backend server. The key stack components are summarized below:

Component	Technology / Spec	Description / Role
Core Render Engine	Three.js (^0.184.0)	Handles WebGL rendering, geometries, neon lighting, fog, and particle layers.
Overlay Badges	CSS2DRenderer	Mounts standard HTML text label badges directly above 3D node coordinates.
Computer Vision	MediaPipe Hands API	Extracts 21 hand skeleton joints from webcam video feed in real-time.
Immersive VR	WebXR Device API	Binds virtual reality rendering and controller/hand tracking sessions.
Frontend Styling	Vanilla CSS Variables	Renders glassmorphic control overlays and sidebar text panels.

Bundler / Server	Vite (^8.0.12)	High-speed build bundler and hot-module reloading dev server.
------------------	----------------	---

3. System Architecture & Modular Layout

The application follows a modular, decoupled architecture where the algorithm engines, the 3D visualizers, the user interfaces, and the gesture controllers interact through structured events and APIs. Below is an overview of the directory hierarchy:

```

downfall/
  ■■■ index.html          # Main entry document containing overlay layouts
  ■■■ package.json        # Dependency manager specifying Vite and Three.js
  ■■■ src/
    ■ ■■■ main.js         # Entry script initializing engine modules
    ■ ■■■ style.css        # Glassmorphic and typographic CSS stylesheets
    ■ ■■■ dsa/            # CORE ALGORITHM ENGINES
      ■ ■■■ index.js       # Exports and combines all engines
      ■ ■■■ bst.js         # Binary Search Tree traversal and insertion
      ■ ■■■ graph.js       # BFS/DFS traversals and node structures
      ■ ■■■ linkedList.js  # Chain nodes, insert/delete at head/index
      ■ ■■■ queue.js       # FIFO Queue insertion and deletion
      ■ ■■■ stack.js       # LIFO Stack push/pop operations
      ■ ■■■ sorting.js     # Bubble, Quick, Merge Sort and manual sorting
    ■ ■■■ interaction/    # USER CONTROL SYSTEMS
      ■ ■■■ handSim.js     # Keyboard/Mouse fallback glove simulator
      ■ ■■■ webcamHands.js # MediaPipe webcam keypoint tracker
      ■ ■■■ webxr.js       # Immersive WebXR VR headset connector
    ■ ■■■ ui/             # HUD OVERLAYS & CONTROLS
      ■ ■■■ hud.js         # Binds HUD buttons, command inputs, and timelines
      ■ ■■■ keyboard.js    # On-screen keyboard for gesture text inputs
    ■ ■■■ visualization/  # THREE.JS WEBGL SYSTEMS
      ■ ■■■ animator.js    # Node animations, levitations, and transitions
      ■ ■■■ engine.js      # WebGL renderer, OrbitControls, and lights
      ■ ■■■ environment.js # Starfields, holographic floor, and drift particles

```

Component Relationships

- 1. Main Loop (main.js):** Initializes the VisualEngine, ClassroomEnvironment, and SceneAnimator. Instantiates both the DesktopHandSimulator (physical controls) and WebcamHandsManager (gesture controls). It establishes the animation loop using `renderer.setAnimationLoop()` to ensure WebXR compatibility.
- 2. Visual Engine (engine.js):** Manages canvas containers, ambient and spotlight systems, fog levels, and orbit camera lerps. It runs both the WebGL canvas renderer and the CSS2DRenderer simultaneously.
- 3. Scene Animator (animator.js):** Acts as a bridge between the abstract state of data structures and their visual representation in 3D. It manages position lerping, object levitation, dynamic node colors, and the instantiation of 3D lines/arrows representing pointers or graph edges.
- 4. HUD Controller (hud.js):** Captures clicks on sidebar elements, handles scrubbers, command-line inputs, and speed sliders. On state change, it asks the active dsa engine to generate animation steps, which are then passed to the SceneAnimator to render.

4. DSA Engine Mechanics & Step Simulation

Each data structure engine (located in `src/dsa/`) operates by receiving an action command (e.g. *push*, *insert*, *sort*) and generating an array of **animation step states**. Instead of executing the entire algorithm instantly, the engine returns a chronological sequence of steps, where each step defines:

- **Nodes/Items:** The current state (value, coordinates, color code) of all nodes in the scene.
- **State Markers:** Specific labels marking nodes as *spawn*, *active*, *highlighted*, or *delete_lift*.
- **Pseudocode Highlights:** Indices of lines in the algorithm's pseudocode that correspond to the active execution step.
- **Text Explanation:** Context-rich explanations displayed on the HUD describing the current operation.

Stack & Queue Engines

The **Stack Engine** (`stack.js`) manages a Last-In, First-Out (LIFO) stack. It generates steps for Push (spawning an element and moving it to the top of the tower) and Pop (lifting the top element, fading it out, and deleting it). The **Queue Engine** (`queue.js`) operates a First-In, First-Out (FIFO) queue, lining nodes up horizontally, where elements enter from the right (rear) and exit from the left (front).

LinkedList Engine

The **LinkedList Engine** (`linkedList.js`) allows index-based insertion and deletion. For middle-of-chain operations, it generates traversal steps that highlight each node sequentially (highlight state: *active/highlighted*), links the new node's next pointer to the successor, and then breaks and re-links the predecessor node to point to the new node in 3D.

Binary Search Tree (BST) Engine

The **BST Engine** (`bst.js`) maps integer keys to a recursive parent-child tree hierarchy. Operations include Insertion (animating traversal from the root, branching left/right based on value comparisons) and Tree Traversals (In-order, Pre-order, Post-order), where a "scanning light" animates through nodes following recursive visit orders.

Graph & Sorting Engines

The **Graph Engine** (`graph.js`) manages nodes in 3D spaces with connecting edge vectors. It simulates Breadth-First Search (BFS) and Depth-First Search (DFS) traversals by pushing queue/stack updates and coloring visited nodes. The **Sorting Engine** (`sorting.js`) represents arrays as vertical columns. It supports Bubble, Quick, and Merge Sorts. In addition, it allows users to manually grab columns using webcam hand gestures or simulated hands, sorting them manually. If successfully sorted, a victory condition triggers, flashing columns in celebration green.

5. Step-by-Step Local Setup & Execution Guide

Follow these steps to set up, run, configure, and compile the Spatial DSA Tutor on a local machine.

Prerequisites

Before starting, ensure that **Node.js** (v18.0.0 or higher recommended) is installed on your operating system.

Step 1: Install Dependencies

Open a terminal window in the project's root folder (containing package.json) and run the following command to download Vite, Three.js, and other necessary assets:

```
npm install
```

Step 2: Start Development Server

To start the local hot-reloading development server, run the following npm script. This spins up the server, typically bound to `http://localhost:5173/`:

```
npm run dev
```

Open the output URL in a web browser. Ensure you grant camera permissions if prompted to enable real-time webcam hand tracking.

Step 3: Compile for Production

To bundle and compile all JavaScript, CSS, and HTML assets into a highly optimized, minified production build, run:

```
npm run build
```

This command outputs all compiled assets to the `dist/` directory, which can be deployed to static web hosts such as Netlify, Vercel, or GitHub Pages.

Step 4: Local Preview of Build

To preview the compiled assets locally using a lightweight server, run the following command:

```
npm run preview
```

6. Interactive Interface Guide & Control Commands

Once the application is running, users can interact with the system using three different command methods:

A. Command-Line Console Command Guide

A command bar interface (visible on the overlay HUD) allows power users to type textual operations to interact with active labs. The list of supported commands includes:

Topic	Command Example	Execution Effect
Stack	push 42	Spawns a new node labeled 42 and pushes it to the stack top.
Stack	pop	Pops and animates deletion of the top node.
Queue	enqueue hello	Enqueues a node labeled 'hello' into the queue rear.
Queue	dequeue	Dequeues the front element of the queue line.
LinkedList	insert a 2	Inserts a node labeled 'a' at index 2 of the list chain.
LinkedList	delete 1	Deletes the node located at index 1.
BST	create tree with 10 5 15	Wipes current BST and inserts 10 (root), 5, and 15.
BST	show inorder	Starts inorder tree traversal, lighting nodes in order.
Graph	show bfs	Runs Breadth-First Search traversal from starting node 'A'.
Sorting	sort bubble	Triggers step-by-step Bubble Sort algorithm visualization.
General	clear	Wipes all active data structure meshes from the 3D scene.

B. Keyboard Shortcuts

- **Space (Tap):** Play or pause step-by-step playback of simulation timelines.
- **ArrowRight:** Advance one step forward in the simulation timeline.
- **ArrowLeft:** Move one step backward in the simulation timeline.
- **Escape:** Exit the active laboratory view and return to the main dashboard.
- **Space (Hold) + Mouse Drag:** Move a simulated right hand (wireframe cyber-glove). Left-click to pinch/grab nodes.
- **Shift + Space (Hold) + Mouse Drag:** Move a simulated left hand to enable dual-hand gesture operations.

C. Webcam Hand Gesture Specifications

- **Single-Hand Pinch:** Brings thumb and index fingertips closer than 0.38x hand scale. Grabs a node if hovered, allowing drag-and-drop. Pinching over UI controls clicks buttons or drags sliders.
- **Single-Hand Release:** Spreads thumb and index wider than 0.60x. Releases grabbed items, initiating inertial glide and layout snapping.
- **Left-Hand Dolly Zoom:** Pinch left thumb/index in empty space. Move hand closer to webcam to zoom in, and pull back to zoom out.
- **Dual-Hand Scale/Rotate:** Pinch both hands in empty space. Move hands apart or closer together to scale the data structure mesh; rotate hands relative to each other to rotate the data structure.